

PG4200
Candidate: 18



PG4200: Algorithms and Data Structures

Kristiania University College, Oslo, Norway

Prof. Dr. Rashmi Gupta

Candidate: 18

Question 1	2
Description of the task:.....	3
Solution:	3
Question 2.....	4
Description of the task:.....	5
Solution:	5
Complexities:	5
Code:.....	5
Question 3.....	6
Description of the task:.....	7
Solution:	7
Question 4.....	8
Description of the task:.....	9
Solution:	9
Question 5.....	10
Description of the task:.....	11
Solution:	11
Code:.....	11
Question 6.....	13
Question 1 What is Complexity Class? Why is it known as complexity classes in data structures?	14
Question 2 Why do we use Big O Notation to analyse algorithms' complexities in data structures?	14
Question 3 What is P class, and What are the features of this class? Provide a real-life example of this type of problem	14
Question 4 What is NP class, and What are the features of this class? Provide a real-life example of this type of problem	15
Question 5 What is an NP-complete class, and What are the features of this class? Provide a real-life example of this type of problem	15

Sources	16
Question 1 sources	17
Question 2 sources	17
Question 3 sources	17
Question 4 sources	17
Question 5 sources	18
Question 6.....	18

Question 1

Description of the task:

For this task, I will show how to access any random element from an one-dimensional array. The given base adress of an array is A [1300 ... 1900], and the size of each element is 2 bytes in the memory. I will now demonstrate how to find the address of A [1704].

Solution:

To locate an element in a dimensional array, one must first figure out if the array is one-dimensional, two-dimensional, or three-dimensional. In this case, the array is one dimensional. A 1 dimensional array or if preferred, a single dimension array is a type of linear array. To access an element, one must involve a single subscript that can either represent a row or column index (GeeksforGeeks, 2025). Because the array is stored as one continuous block of memory and every element occupies the same fixed size, that means that any element can be reached directly through a calculation rather than by going one by one through the array. Think of it as RAM (Random access memory) in a way (Kusmagul,2024).

Now to calculate the address of A [1704] one must utilize this following formula:

Address of A [index] = $B + W * (\text{Index} - \text{LB})$

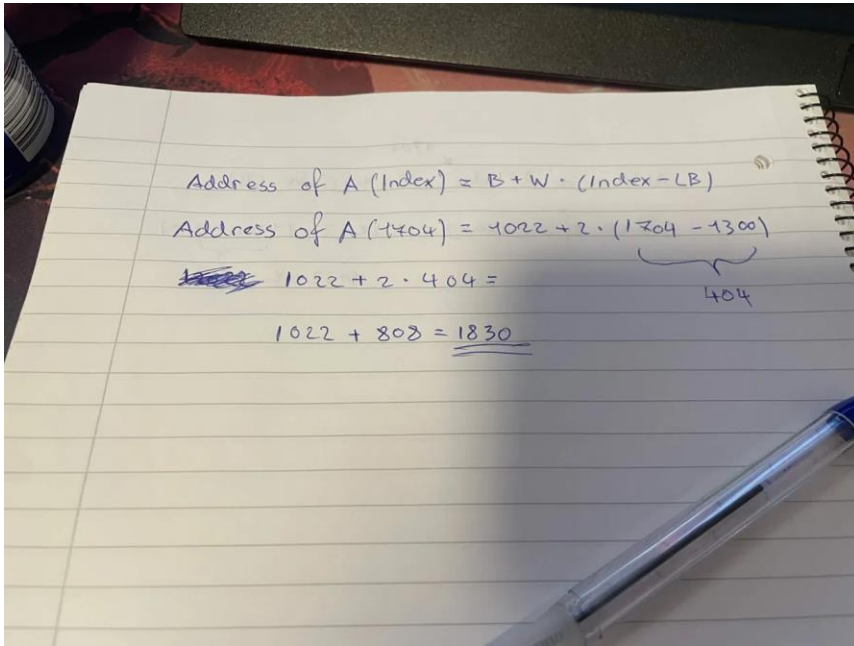
B = Base Address of the array or where the array begins in memory

W = Storage size of one element in bytes

LB = Lower bound of index

Index = the element whose address need to be found

(GeeksforGeeks, 2025).



Address of A (Index) = $B + W \cdot (\text{Index} - LB)$
Address of A (1704) = $1022 + 2 \cdot (1704 - 1300)$
~~1022~~ $1022 + 2 \cdot 404 =$ 404
 $1022 + 808 = \underline{1830}$

The image shows a handwritten calculation on a piece of lined paper. The formula for the address of an element in an array is given as $\text{Address of A (Index)} = B + W \cdot (\text{Index} - LB)$. Then, the address of A[1704] is calculated as $1022 + 2 \cdot (1704 - 1300)$. The calculation is simplified to $1022 + 2 \cdot 404$, where 404 is the result of $1704 - 1300$. The final result is $1022 + 808 = 1830$, which is underlined.

The outcome of the calculation is that A [1704] is at the address 1830

Question 2

Description of the task:

in the following task, the student will implement a stack with the following operations, a push, a pop, and getMin. These methods do the following operations. The getMin method returns the smallest value that is in the stack currently. If the stack is empty, it returns value -1 . The next methods are essential in every stack, and that is a push and pop method. Every stack performs a push and pop in constant time $O(1)$, so the only challenge is to make the getMin run in $O(1)$ as well. The only specification that is required is to set the limit of the input to 100 queries with the values between 1 and 100.

Solution:

The most obvious way to find the minimum would be to look through every element in the stack each time getMin is called. This could work, but the result was too slow. Scanning every element will give getMin a time complexity of $O(n)$ which violates the requirements of the task.

To avoid this problem one must implement a second, parallel stack alongside the main one, often called an auxiliary stack. Its job is to remember for every level of the main stack what the smallest value is at that point. The push and pop operations are arranged so that the top of the auxiliary stack always holds the current minimum (GeeksforGeeks, 2026). On each push, the new minimum is simply the smaller value of the one being pushed and the previous minimum before that again, so it can be recorded in one comparison. On each pop, both stacks mimic each other's shrinking so that the auxiliary stack's top shows the minimum of whatever remains. That means that getMin only needs to read the top of the auxiliary stack instead of traveling through the whole stack (GeeksforGeeks, 2026).

Complexities:

With the design of push pop and getMin each method does a small amount of work. That means that all three methods run in $O(1)$ time complexity. The only downside is the cost of memory the auxiliary stack doubles the space used, giving $O(n)$ space complexity overall (Ganesh, 2009). There is another advanced variant mentioned by geeks for geeks that avoids the extra stack by encoding the minimum into the stored values with arithmetic, achieving $O(1)$ space. The problem is flimsy, harder to read, and risks overflow, so the two stacks are preferably better (GeeksforGeeks, 2026).

Code:

```
//You are given N elements , and your task is to Implement a Stack in which you can get a
minimum element in O(1)
//time. You are required to complete the three methods:
//o push() which takes one argument, an integer 'x' to be pushed into the stack,
//o pop() which returns an integer popped out from the stack, and
//o getMin() which returns the min element from the stack. (-1 will be returned if for
pop() and getMin() the
//stack is empty.)
//o Constraints:
//1 <= Number of queries <= 100
//1 <= values of the stack <= 100

import java.util.Scanner;

public class task2stack {

    int [] stack; //hold the stacks values
    int [] minOfStack; // minimum of stack
    int topOfStack; // top of the stack

    public task2stack(){
        int capacity = 100; //max pushes no more pushes can be done
        stack = new int[capacity];
        minOfStack = new int[capacity];
        topOfStack = -1; // shows the stacks empty
    }

    //small helper method that helps the program from crashing
    public int remaining () {
        return stack.length - (topOfStack + 1);
    }

    // the push method
    public void push(int a){

        //simple helper if statment to help from crashing
        if (topOfStack == stack.length-1){
            System.out.println("Stack is full" + a);
            return;
        }
    }
}
```

```
        topOfStack++;
        stack[topOfStack] = a;

        if (topOfStack == 0) {
            minOfStack[topOfStack] = a;
        } else {
            minOfStack[topOfStack] = Math.min(minOfStack[topOfStack - 1], a );
        }
    } //end of push

    //pop method
    public int pop(){
        if (topOfStack == -1){
            System.out.println("Stack is empty");
            return -1;
        }
        int a = stack[topOfStack];
        topOfStack--;
        return a;
    } // end of pop method

    //returns the minimum element. If the stacks empty it returns -1
    public int getMin(){
        if (topOfStack == -1){
            System.out.println("Stack is empty");
            return -1;
        }
        return minOfStack[topOfStack];
    }

    //helper class for the menu. just prints the current stack
    //not needed just a fun addon that i wanted to add :)
    public void printStack(){
        if (topOfStack == -1){
            System.out.println("Stack is empty");
            return;
        }
        System.out.println("Stack:");
        for (int i = 0; i <= topOfStack; i++) {
            System.out.print(stack[i] + " ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
```

```
task2stack stack = new task2stack();
Scanner input = new Scanner(System.in);

System.out.printf("Welcome to my super duper cool stack\n");

while (true){
    System.out.println("Super crazy cool menu");
    System.out.println("1. push");
    System.out.println("2. pop");
    System.out.println("3. minimum");
    System.out.println("4. printStack");
    System.out.println("5. exit");

    if (!input.hasNext()){
        System.out.println("enter a number from 1 to 5");
        input.next();
        continue;
    }

    int choice = input.nextInt();

    //choice 1
    if (choice == 1){
        System.out.println("Enter element to be pushed into the stack");
        int value = input.nextInt();
        stack.push(value);
        System.out.println("Pushed: " + value + ". Slots left: " +
stack.remaining());
    }
    // choice 2
    else if (choice == 2){
        int popped = stack.pop();
        if (popped == -1){
            System.out.println("Stack is empty");
        } else {
            System.out.println("Popped: " + popped + ". Slots left: " +
stack.remaining());
        }
    }
    // choice 3
    else if (choice == 3){
        int min = stack.getMin();
        if (min == -1){
            System.out.println("-1");
        } else {
            System.out.println("Minimum: " + min);
        }
    }
}
```

```
    }  
    // choice 4  
    else if (choice == 4){  
        System.out.println("Here is the stack!! :D");  
        stack.printStack();  
    }  
    // choice 5  
    else if (choice == 5){  
        System.out.println("Goodyeee (-.-)");  
        break;  
    } else {  
        break;  
    }  
}  
  
}  
  
}
```

Question 3

Description of the task:

For this task, I must delete a certain node in this case node 60 from the Binary Search Tree. Later, I will provide a step-by-step algorithm and a way of solving the problem. It will follow a detailed explanation of the algorithm and a detailed explanation of the step-by-step process of the drawing.

Solution:

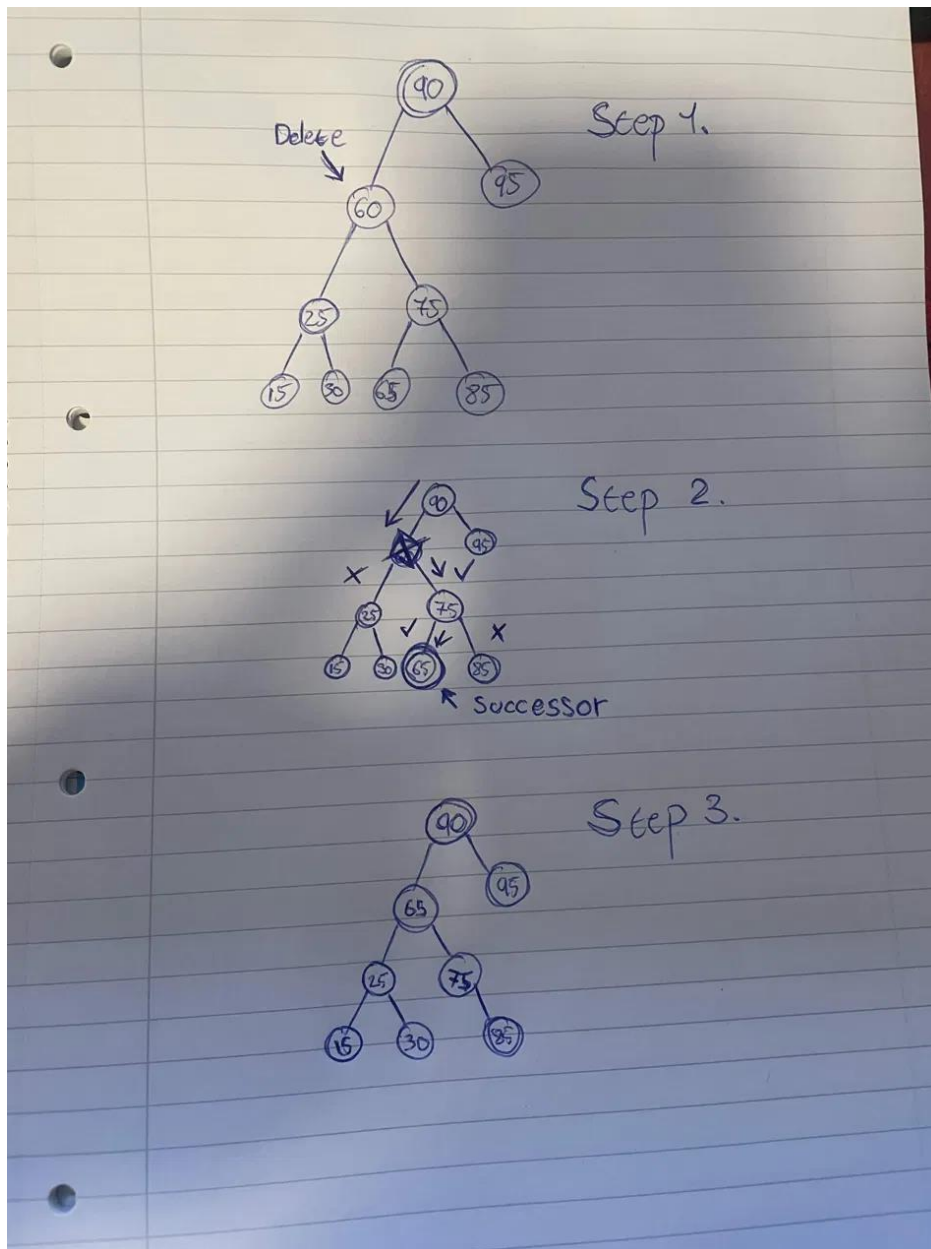
When a node is deleted, which in this case is node 60, a worthy successor must be found. A binary search tree deletion has three cases depending on how many children the node has. No children? Remove it. If the tree has one child? Connect the child to the parent. And if it has two children? Replace the node with its in-order successor then delete that successor from its original spot (GeeksforGeeks, 2025). Node 60 has two children, so it falls under the two children categories.

Here is a demonstration on paper on how I have completed this task:

First 1. I drew out the binary search tree and deleted the node requested of me.

Step 2. After deleting node 60 I needed to find a worthy successor, which is the smallest value in 60's right subtree. I went right in the 60's subtree which starts at 75. From 75 I went left as far as I could, in this case it was only 65. That means the worthy successor was 65, because it did not have any children and was the smallest value in 60's right subtree.

Step 3. The tree is now finished, and what remains is the final answer to this task.



Question 4

Description of the task:

For this task I shall perform a Merge Sort of the following Array = {70, 50, 30, 10, 20, 40, 60}. I will draw a step-by-step tutorial to show and document the divide and conquer strategy of merge sort. I will also provide a detailed explanation of the algorithm and the graphical representation.

Solution:

For this task, I will implement a detailed step-by-step merge sort operation on paper and explain the algorithm both visually and theoretically. Merge sort works by divide and conquer. The algorithm keeps splitting the array in half until every piece is a single element. Since a single element is already sorted on its own, there is nothing more to divide (Gupta, 2026, p. 9). After the divide then the pieces are merged back together two at a time, and each merge compares the front element of each piece and takes the smaller one into the merged list, so the combined piece comes out in sorted order. Repeating this all the way back up produces the fully sorted array (GeeksforGeeks, 2025 and W3Schools, n.d.).

Here is my pen and paper step by step explanation:

The first phase is the divide phase with the following steps.

Step 1. draw out the full array: 70 50 30 10 20 40 60 then draw out the split

Step 2. Split into two halves 70 50 30 and 10 20 40 60

Step 3. Split the halves again 70 and 50, 30, then 10, 20 and 40, 60

Step 4. Split the halves into single numbers 70, 50, 30, 10, 20, 40, 60

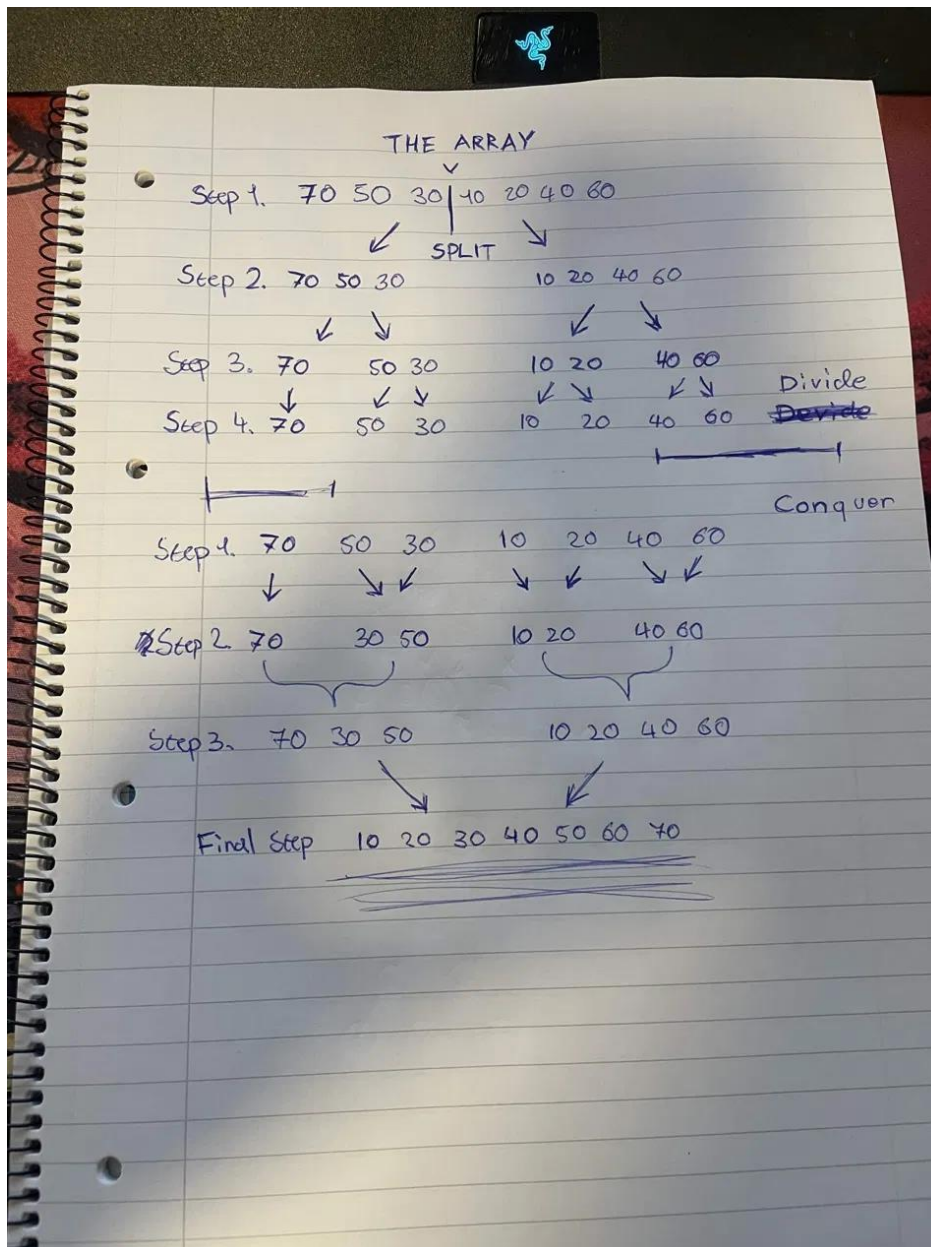
Then over to the second phase, which is the merge phase or conquer phase.

Step 1. Merge the singled-out numbers into sorted pairs. 50 and 30 become 30, 50, then 10 and 20 stay as 10, 20, then 40 and 60 stay 40, 60. Then 70 stays and waits because it has no partner

Step 2. Merge those pairs and the single into sorted groups. 70 joins 30 and 50, then 10 and 20 join 40 and 60.

Step 3. Final merge of the two sorted halves 30, 50, 70 and 10, 20, 40, 60. Now compare each number one by one and take the smaller one first.

Final step. And the final step should be 10, 20, 30, 40, 50, 60, 70



Question 5

Description of the task:

In this task I will code, explain and demonstrate difference between these two algorithms, to show the importance of developing efficient algorithms:

Recursive algorithm to solve fib(n) for $1 \leq n \leq 5$ Fibonacci term.

Iterative algorithm to solve fib(n) for $1 \leq n \leq 5$ Fibonacci term.

Solution:

The code is provided with this report. This is just the conclusion of the code

Both algorithms return the same Fibonacci values of 1,1,2,3,5 for $n=1$ to 5, so that means they are both correct. Now the big difference is in which is more efficient and does less work. Since when it comes to computing, efficiency is everything, so even the smallest differences matter.

The recursive version is slower because it recomputes the problems over and over again. To calculate fib (5) it must calculate fib (4) and fib (3), but fib (4) calculates fib (3) again, and so on and so on until fib (1). This happens because nothing is remembered between calls, the same values are computed many times (Marshall,2024). This gives it exponential time complexity, with the recurrence $T(n) = T(n-1) + T(n-2) + O(1)$ resolving to roughly $O(2^n)$, and a tighter bound of about $O(1.618^n)$ (GeeksforGeeks, 2025a)

When it comes to the iterative version, computes each term only once, from the bottom up, keeping just the previous two values. It runs in linear $O(n)$ time and constant $O(1)$ space, which is a clear improvement over the recursive approach (Baeldung, 2026).

My code solidifies this even further. For $n = 5$, the recursive method already needs 9 calls against the iterative method's 3 loops, and for fib (30) the gap explodes to 1,664,079 recursive calls to only 28 iterative loops for the exact same answer. This shows that even though both algorithms are correct, the iterative one is far more efficient in both time and memory, which shows why efficiency is far more important even though it gives the same result.

Code:

```
//Regardless of how fast computers become or
//how cheap memory gets, efficiency will always remain an important
//consideration. To show the importance of developing efficient algorithms,
//compare these two algorithms:
//Recursive algorithm to solve
//fib(n) for  $1 \leq n \leq 5$  Fibonacci term
//Iterative algorithm to solve
//fib(n) for  $1 \leq n \leq 5$  Fibonacci term
//Justify which algorithm is more efficient and why through the solution with code.

//https://www.baeldung.com/cs/fibonacci-computational-complexity

//https://www.geeksforgeeks.org/dsa/time-complexity-recursive-fibonacci-program

//https://www.geeksforgeeks.org/dsa/fibonacci-series/

//https://stackoverflow.com/questions/39839640/java-fibonacci-sequence-fast-method

public class task5fibo {

    // counts how many times the method has been called upon
    static int recursiveCalls = 0;

    // Recursive Fibonacci
    public static int fibRecursice(int n) {
        recursiveCalls++;
        if (n <= 2) {
            return 1;
        }
        return fibRecursice(n - 1) + fibRecursice(n - 2);
    }

    // Iterative Fibonacci
    public static int fibIterative(int n) {
        if (n <= 2) {
            return 1;
        }
        int previous = 1;
        int current = 1;
        for (int i = 3; i <= n; i++) {
```

```
        int next = previous + current;
        previous = current;
        current = next;
    }
    return current;
}

public static void main(String[] args) {
    for (int i = 1; i <= 5; i++) {
        recursiveCalls = 0;
        int r = fibRecursice(i);
        int c = recursiveCalls;
        int it = fibIterative(i);
        int loops = (i <= 2) ? 0 : i - 2;
        System.out.println("n, " + i + " | fib(n), " + r + " | recursive calls, " + c
            + " | iterative loops, " + loops + " | fibIterations, " + it);
    }
    recursiveCalls = 0;
    fibRecursice(30);
    System.out.println("\nFor fib(30):");
    System.out.println("recursive calls = " + recursiveCalls);
    System.out.println("iterative calls = " + (30 - 2));
}
}
```

Question 6

Question 1 What is Complexity Class? Why is it known as complexity classes in data structures?

A complexity class is a group of computational problems that all require a similar amount of computing resources to solve. The two resources that matter here are time complexity, how many steps the algorithm takes, space complexity, how much memory it needs and problems that demand comparable amounts of each can be sorted into the same class (Gupta, 2026b, p.3 and GeeksforGeeks, 2025b).

A complexity class is called a complexity class because the grouping is based on computational complexity this means how hard a problem is to resolve measured in time and space, rather than what the problem actually is about. This lets people organize different problems by difficulty and time about a whole group at once instead of analyzing each problem (GeeksforGeeks, 2025b). Common complexity classes include but are not limited to P class (Polynomial time), NP class (Non-deterministic Polynomial Time), Co-NP class (complement of NP class), NP-hard Class and NP-Complete Class (Gupta, 2026b, p.4)

Question 2 Why do we use Big O Notation to analyse algorithms' complexities in data structures?

Big O notation shows how an algorithm use of resources grows as the input size is increased. We rely on it because it captures the things that actually matters for performance. For example, how an algorithm behaves as the input grows instead of measuring exact seconds (GeeksforGeeks, 2026a). Measuring seconds is not very reliable because you can get different outputs when a person uses different machines. Big O ignores hardware and focuses only on growth rates. That means it compares the algorithm itself instead of the computer running the algorithm. This is why an $O(n)$ algorithm will always be faster than an $O(n^2)$ one on larger inputs no matter how fast or slow the hardware is. It is also normally used to express the worst case scenario, giving the information that the algorithm will never be slower than that (Gupta, 2026a, p18).

Question 3 What is P class, and What are the features of this class?

Provide a real-life example of this type of problem

P stands for Polynomial time. It is a collection of decision problems this means problems with only a yes or no answer, that can be solved by a deterministic machine in polynomial time. This means that the running time grows as a polynomial of the input size such as n , $n \log n$, or n^2 rather than exponentially (Gupta, 2026b, p.6 and GeeksforGeeks, 2025b).

The feature of a P class is that the solution is easy to find and that the problems are tractable meaning solvable both in theory and in practice, so they are easy to both solve and verify (Gupta, 2026b, p 6-7). A real life example of this could be a GPS finding the shortest route between two points using Dijkstra's algorithm (Gupta, 2026b, p7).

Question 4 What is NP class, and What are the features of this class?

Provide a real-life example of this type of problem

NP stands for non-deterministic polynomial time. It is the collection of decision problems whose solutions can be verified in polynomial time, even though actually finding the solution might take a very long time. A non-deterministic machine could solve such a problem in polynomial time by guessing the answer and then checking it (Gupta, 2026b, p.8).

Its main feature with NP class is that these problems are hard to solve but easy to check. The real-life problem of such classes is a Sudoku puzzle. Such puzzles are hard to solve but easy to check if they are completed (Gupta, 2026b, p8 – 9). There is also The Traveling Salesman Problem, where finding the shortest route through all the cities is hard, but checking whether the given route stays under a certain distance is easy (Gupta, 2026b, p9).

Question 5 What is an NP-complete class, and What are the features of this class? Provide a real-life example of this type of problem

NP-complete problems are the hardest problems in NP. NP-complete problems are a part of a larger class of NP problems. GeeksforGeeks explains that NP problems are a class of computational problems that can be solved in polynomial time by a non-deterministic machine and can be verified in polynomial time by a deterministic machine (GeeksforGeeks, 2026b).

The unique feature of NP-complete class is that if anyone ever found a polynomial-time solution to just one NP-complete problem, that method could be used to solve every NP

problem in polynomial time (GeeksforGeeks, 2025b and Gupta, 2026b, p.12). A real-life example of NP-complete class could be a scheduling algorithm. For example, the conflicts that arise with day to day problems like deciding on exam dates or organizing important events (GeeksforGeeks, 2025a). Or we could use the Travelling Salesman Problem since the transportation sector of EU makes up 10% of its GDP. Being able to find the most efficient path between start and finish would make travel and delivery very easy and would save a lot of money (GeeksforGeeks, 2025a).

Sources

Question 1 sources

Kusmagul, N. (2024, September 25). Why are arrays so fast?

Faangshui, <https://blog.faangshui.com/p/why-are-arrays-so-fast>

GeeksforGeeks. (2025, July 23). Calculation of address of element of 1-D, 2-D, and 3-D using row-major and column-major order. GeeksforGeeks, <https://www.geeksforgeeks.org/dsa/calculation-of-address-of-element-of-1-d-2-d-and-3-d-using-row-major-and-column-major-order/>

Question 2 sources

(Used in text)

Ganesh M. (2009, March 26). Design a stack such that getMinimum() should be O(1) Stack Overflow, <https://stackoverflow.com/questions/685060/design-a-stack-such-that-getminimum-should-be-o1>

GeeksforGeeks. (2026, April 3). Stack with Minimum. GeeksforGeeks, <http://geeksforgeeks.org/dsa/design-a-stack-that-supports-getmin-in-o1-time-and-o1-extra-space/>

(Used in code)

GeeksforGeeks. (2026, July 23). Stack Class in Java. GeeksforGeeks, <https://www.geeksforgeeks.org/java/stack-class-in-java/>

Question 3 sources

GeeksforGeeks. (2025, December 8). Deletion in Binary Search Tree (BST). GeeksforGeeks, <https://www.geeksforgeeks.org/dsa/deletion-in-binary-search-tree/>

Gupta, R. (2026). LO 2_SearchingAlgorithms. PG4200 Algorithms and Data Structures, Kristiania University College.

Question 4 sources

GeeksforGeeks. (2025, October 3). Merge sort. GeeksforGeeks, <https://www.geeksforgeeks.org/dsa/merge-sort/>

Gupta, R. (2026). LO 3_SortingAlgorithms. PG4200 Algorithms and Data Structures, Kristiania University College.

W3Schools. (n.d.). DSA Merge sort. W3Schools,
https://www.w3schools.com/dsa/dsa_algo_mergesort.php

Question 5 sources

Baeldung. (2026, January 4). Fibonacci series in Java. Baeldung,
<https://www.baeldung.com/java-fibonacci>

Burakkaanerce. (2016, October 3). Java Fibonacci Sequence fast method.
Stack Overflow, <https://stackoverflow.com/questions/39839640/java-fibonacci-sequence-fast-method>

GeeksforGeeks. (2025, August 27a). Time complexity of recursive Fibonacci program.
GeeksforGeeks, <https://www.geeksforgeeks.org/dsa/time-complexity-recursive-fibonacci-program>

GeeksforGeeks. (2025, December 10b). Fibonacci Sequence.
GeeksforGeeks, <https://www.geeksforgeeks.org/maths/fibonacci-sequence/>

GeeksforGeeks. (2026, May 28). Program to Print Fibonacci Series.
GeeksforGeeks, <https://www.geeksforgeeks.org/dsa/fibonacci-series/>

Marshall, E. (2024, March 18). Computational Complexity of Fibonacci Sequence.
Baeldung, <https://www.baeldung.com/cs/fibonacci-computational-complexity>

Question 6

GeeksforGeeks. (2025a, July 23). Real-world applications of a constructive $P = NP$ proof.
GeeksforGeeks, <https://www.geeksforgeeks.org/theory-of-computation/real-world-applications-of-a-constructive-pnp-proof/>

GeeksforGeeks. (2025b, July 23). P, NP, CoNP, NP hard and NP complete | Complexity Classes
GeeksforGeeks, <https://www.geeksforgeeks.org/dsa/types-of-complexity-classes-p-np-conp-np-hard-and-np-complete/>

GeeksforGeeks. (2026a, March 28). Big O notation.
GeeksforGeeks, <https://www.geeksforgeeks.org/dsa/analysis-algorithms-big-o-analysis/>

GeeksforGeeks. (2026b, March 16). Introduction to NP-Complete Complexity classes. GeeksforGeeks, <https://www.geeksforgeeks.org/dsa/introduction-to-np-completeness/>

Gupta, R. (2026a). LO 5_PG4200_29 Jan. Learning Outcome 5. PG4200 Algorithms and Data Structures, Kristiania University College.

Gupta, R. (2026b). LO 6_Computability and Complexity. Learning Outcome 6. PG4200 Algorithms and Data Structures, Kristiania University College.

Khan Academy. (n.d.). Big-O notation.

Khan Academy, <https://www.khanacademy.org/computing/computer-science/algorithms/asymptotic-notation/a/big-o-notation>